



Universidad
Nacional
de Córdoba



Facultad de Matemática,
Astronomía, Física y
Computación

Trabajo Práctico Especial

MODELOS Y SIMULACIÓN - 2025

Ruiz Luciana Gimena, Nesteruc Tiago

15 de Junio de 2025

Resumen

En este trabajo se estudian y comparan tres métodos de generación de números pseudoaleatorios uniformes: el Generador Congruencial Lineal (LCG), el XORShift y el Mersenne Twister (MT19937). Se evalúa su calidad estadística y se aplican luego a la simulación de un sistema de colas de un solo servidor, con llegadas no homogéneas modeladas por un proceso de Poisson con tasa variable en el tiempo, y tiempos de servicio exponenciales.

El objetivo principal es analizar cómo influyen las características estadísticas de los generadores en los resultados de una simulación estocástica realista. Para esto, se midieron distintas características del sistema, como cuánto tiempo está ocupado el servidor, los tiempos de espera de los clientes y cómo cambia la cola a lo largo del tiempo.

Al comparar los resultados entre los generadores, se observa que aunque todos presentan un desempeño aceptable en general, la elección del generador puede influir bastante cuando la simulación necesita buena calidad en la aleatoriedad.

1. Introducción

En el campo de la simulación estocástica, la generación de números pseudoaleatorios cumple un rol fundamental. La mayoría de los modelos computacionales que representan fenómenos aleatorios dependen de la calidad de estos generadores para producir resultados fiables. Si bien existen numerosos algoritmos para producir secuencias de números pseudoaleatorios, no todos ofrecen el mismo rendimiento en términos de eficiencia computacional ni calidad estadística.

Este trabajo tiene como propósito comparar tres generadores : el Generador Congruencial Lineal (LCG), el XORShift y el Mersenne Twister (MT19937). Para comparar los generadores, se eligió como caso de estudio la simulación de un sistema de colas con un solo servidor. En este modelo, los clientes llegan aleatoriamente con una tasa que varía a lo largo del día y son atendidos con tiempos de servicio distribuidos exponencialmente.

A lo largo de este trabajo se explican las características teóricas de cada generador, la metodología, los resultados obtenidos y el análisis del rendimiento de cada uno, haciendo foco en cómo afectan a una simulación computacional.

2. Descripción teórica de los generadores

2.1. Generador congruencial lineal (LCG)

El **Generador Congruencial Lineal (LCG)** es uno de los métodos más simples y eficientes para generar números pseudoaleatorios. Dado un módulo $M \geq 2$, genera una sucesión de enteros $y_1, y_2, \dots, y_n, \dots$ en el rango $(0, 1, \dots, M - 1)$ mediante la fórmula:

$$y_{n+1} = (ay_n + c) \bmod M, \quad n \geq 1$$

donde a es el multiplicador, c el incremento, y_0 semilla inicial.

En este trabajo se utilizó la configuración: $a = 16807$, $c = 0$, $M = 2^{31} - 1$.

Este generador tiene un período máximo de $M - 1$ si la semilla es distinta de cero, lo que lo hace adecuado para muchas aplicaciones básicas. Su principal ventaja es su bajo costo computacional, ya que solo requiere una multiplicación y una operación módulo.

Sin embargo, el LCG presenta limitaciones estadísticas, especialmente en simulaciones multivariadas o de alta dimensión, donde pueden surgir patrones o correlaciones no deseadas. Por esta razón, si bien es útil para análisis simples, puede no ser adecuado para aplicaciones más exigentes.

2.2. XORShift

El generador **XORShift** es un algoritmo simple y rápido creado por George Marsaglia en 2003.

Utiliza operaciones bit a bit, principalmente **XOR** ($\wedge$) y desplazamientos de bits ($\ll, \gg$), sin necesidad de multiplicaciones o divisiones costosas. El algoritmo recibe una semilla x (normalmente un entero de 32 a 64 bits), que se actualiza en cada paso mediante una secuencia de operaciones **XOR** y desplazamientos de bits.

La operación **XOR** es una operación lógica bit a bit que compara dos bits y devuelve:

- 1 si los bits son diferentes.
- 0 si los bits son iguales.

Los operadores de desplazamiento de bits son:

- ($\ll$) : Desplaza todos los bits del número hacia la izquierda, agregando ceros por la derecha. Matemáticamente, equivale a multiplicar por potencias de 2.
- »■ ($\gg$) : Desplaza todos los bits del número hacia la derecha, descartando los bits menos significativos. Matemáticamente, equivale a dividir por potencias de 2.

El generador puede alcanzar períodos de hasta $2^n - 1$, donde n es la cantidad de bits del estado. Aunque su rendimiento estadístico suele ser superior al del LCG, puede fallar en algunos tests rigurosos de aleatoriedad, especialmente en versiones de menor tamaño o con parámetros mal elegidos.

2.3. Mersenne Twister (MT19937)

El **Mersenne Twister** es uno de los generadores de números pseudoaleatorios más utilizados en la actualidad.

Fue desarrollado por Makoto Matsumoto y Takuji Nishimura en 1997. Sus características principales son:

- Tiene un período enorme de $2^{19937} - 1$
- Generación eficiente y rápida de números pseudoaleatorios.
- Excelente calidad estadística, superando numerosas baterías de tests de aleatoriedad.
- Es el generador por defecto en Python y NumPy.

3. Descripción del problema

3.1. Planteo general

El objetivo de esta simulación es modelar el comportamiento de un sistema de colas de un solo servidor durante un período continuo de 48 horas. El sistema representa una situación en la cual los clientes arriban de forma aleatoria según un proceso de Poisson no homogéneo en el tiempo, y son atendidos uno por uno en orden de llegada.

3.2. Llegadas de clientes

Los tiempos de arribo de los clientes están generados por un proceso de Poisson no homogéneo con tasa dependiente del tiempo dada por la función:

$$\lambda(t) = 30 + 30 \cdot \sin\left(\frac{2\pi t}{24}\right) \quad (\text{clientes/hora})$$

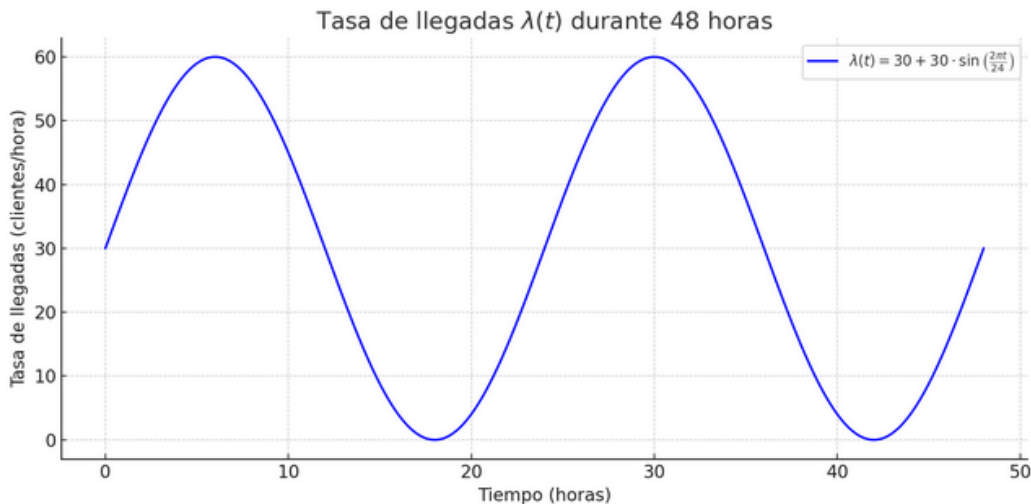


Figura 1: Tasa de llegada $\lambda(t)$ durante un período de 48 horas.

Esta función representa una variación periódica que se repite cada 24 horas, lo cual permite modelar situaciones reales como los cambios en la demanda a lo largo del día (por ejemplo, más clientes en horas pico y menos en horarios nocturnos). La tasa de llegada oscila entre 0 y 60 clientes por hora, con un máximo a las 6:00 y mínimo a las 18:00.

Para simular este proceso de llegadas, se utilizó el método de rechazo adaptado a procesos de Poisson no homogéneos. Este método consiste en generar eventos como si provinieran de un proceso de Poisson homogéneo con tasa constante igual a $\lambda_{\text{máx}}$, y luego aceptar cada uno con probabilidad proporcional a $\frac{\lambda(t)}{\lambda_{\text{máx}}}$.

3.3. Tiempos de servicio

Los tiempos de atención del servidor siguen una distribución exponencial, con tasa de $\mu = 40$ clientes por hora. En promedio, esto implica que cada servicio dura:

$$\frac{1}{\mu} = 0.025 \text{ horas} \quad (\text{equivalente a 1.5 minutos})$$

3.4. Supuestos del sistema

- **Un solo servidor:** sólo un cliente puede ser atendido a la vez.
- **Cola ilimitada:** los clientes pueden esperar el tiempo que sea necesario; no se pierde ninguno.
- **Orden FIFO:** los clientes se atienden en el mismo orden en que llegaron.
- **Servidor siempre activo:** el servidor permanece disponible durante toda la simulación.
- **Sin tiempos muertos:** la atención de un cliente comienza inmediatamente después de finalizar la anterior.

4. Metodología

Para llevar a cabo la simulación del sistema de colas y evaluar el desempeño de los generadores pseudoaleatorios, se siguieron los siguientes pasos organizados en distintas etapas:

4.1. Implementación de los generadores

Se utilizaron tres generadores de números pseudoaleatorios uniformes. En algunos casos, se implementaron desde cero y en otros se utilizaron funciones proporcionadas por bibliotecas.

- LCG

```
1 #Generador 1: LCG
2 def LCG(x, a=16807, c=0, m=2**31 - 1):
3     x = (a * x + c) % m
4     return x / m, x
```

- XORShift

```
1 # Generador 2: XORShift
2 def XORShift(x):
3     x ^= (x << 13) & 0xFFFFFFFF
4     x ^= (x >> 17)
5     x ^= (x << 5) & 0xFFFFFFFF
6     return (x & 0xFFFFFFFF) / 0xFFFFFFFF, x #estandarizo
```

- Mersenne Twister (MT19937)

```

1 # Generador 3: Mersenne Twister
2 # Mersenne Twister
3 def MT():
4     return np.random.rand()# usa MT19937 por defecto

```

4.2. Generación de llegadas

Se utilizó el método de rechazo para simular un proceso de Poisson no homogéneo con tasa $\lambda(t)$.

Se generaron candidatos de llegada a partir de un proceso de Poisson homogéneo con tasa máxima $\lambda_{max} = 60(\text{clientes/hora})$, y se aceptó cada llegada con probabilidad $\frac{\lambda(t)}{\lambda_{max}}$.

```

1 # TASA DE LLEGADAS
2 def lambda_t(t):
3     return 30 + 30 * math.sin(2 * math.pi * t / 24) # t en horas
4
5 def generar_arribos_poisson(rand_func, estado_inicial, T_max=48, lambda_max=60):
6     t = 0
7     arribos = []
8     estado = estado_inicial
9
10    while t < T_max:
11        u , estado = rand_func(estado)
12        t += -math.log(u) / lambda_max
13
14        u2 , estado = rand_func(estado)
15        if u2 <= lambda_t(t) / lambda_max:
16            arribos.append(t)
17
18    return arribos , estado

```

4.3. Generación de Tiempos de Servicio

Los tiempos de atención se generaron usando la distribución exponencial con tasa $\mu = 40(\text{clientes/hora})$.

```

1 # GENERACION DE TIEMPOS DE SERVICIO EXPONENCIAL
2 def generar_tiempo_servicio(rand_func, mu=40):
3     u = rand_func
4     return -math.log(u) / mu

```

4.4. Simulación del sistema

Se simuló un sistema con un solo servidor durante 48 horas.

```

1 #SIMULACION DE SISTEMA DE COLAS
2
3 def simularCola(rand_func, estado_inicial, T_max=48):
4     # Generar los tiempos de llegada

```

```

5     arribos , estado = generar_arribos_poisson(rand_func, estado_inicial, T_max)
6
7     servidor_libre_en = 0
8     clientes = []
9
10    for llegada in arribos:
11        # Determinar cuando comienza el servicio
12        if llegada >= servidor_libre_en:
13            # servidor esta libre
14            inicio = llegada
15        else:
16            # cliente espera
17            inicio = servidor_libre_en
18
19        # Generar tiempo de servicio
20        servicio , estado= generar_tiempo_servicio(rand_func, estado)
21        salida = inicio + servicio
22        servidor_libre_en = salida
23
24        clientes.append({
25            'llegada': llegada, # Tiempo en que el cliente llega al sistema
26            'inicio': inicio,   # Tiempo en que comienza a ser atendido
27            'salida': salida,   # Tiempo en que finaliza el servicio
28            'espera': inicio - llegada, # Tiempo que paso en la cola
29            'tiempo_total': salida - llegada # Tiempo total en el sistema
30        })
31    return clientes

```

5. Resultados y Análisis

En esta sección se presentan los resultados obtenidos a partir de la simulación del sistema de colas.

5.1. Tasa de utilización del servidor por hora

Se analiza el porcentaje de tiempo en el que el servidor estuvo ocupado durante cada hora de la simulación. Este gráfico refleja cómo varía la demanda a lo largo del día, conforme a la función de tasa de llegadas $\lambda(t)$. Las curvas de los tres generadores son similares, mostrando máximos de actividad alrededor de las horas pico (por ejemplo, 6 a.m. y 6 p.m.).

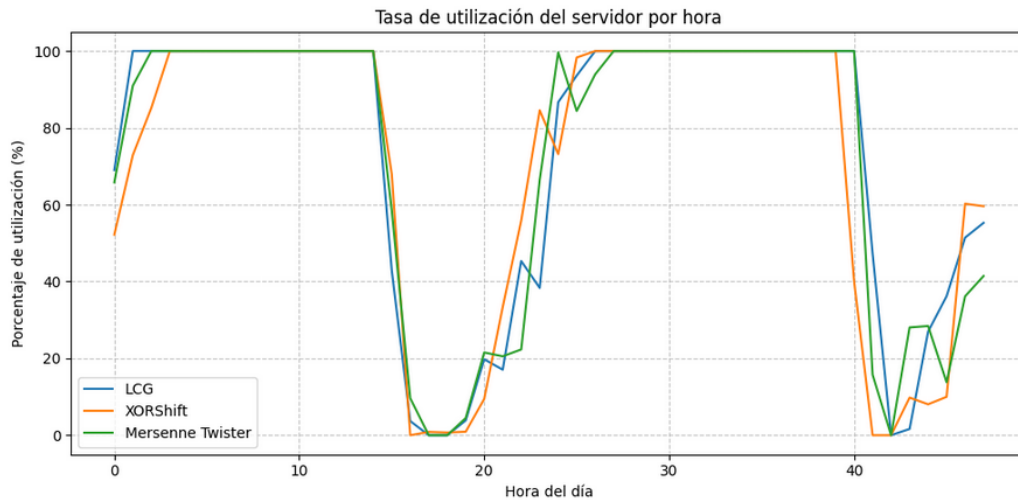


Figura 2: Porcentaje de utilización del servidor a lo largo del tiempo.

5.2. Tiempo promedio en el sistema por cliente

Se calcula cuánto tiempo pasa, en promedio, un cliente desde que llega hasta que se va del sistema. Este valor incluye tanto la espera en la cola como el servicio.

Los tres generadores muestran un comportamiento similar, con ligeras diferencias atribuibles a la calidad de la aleatoriedad.

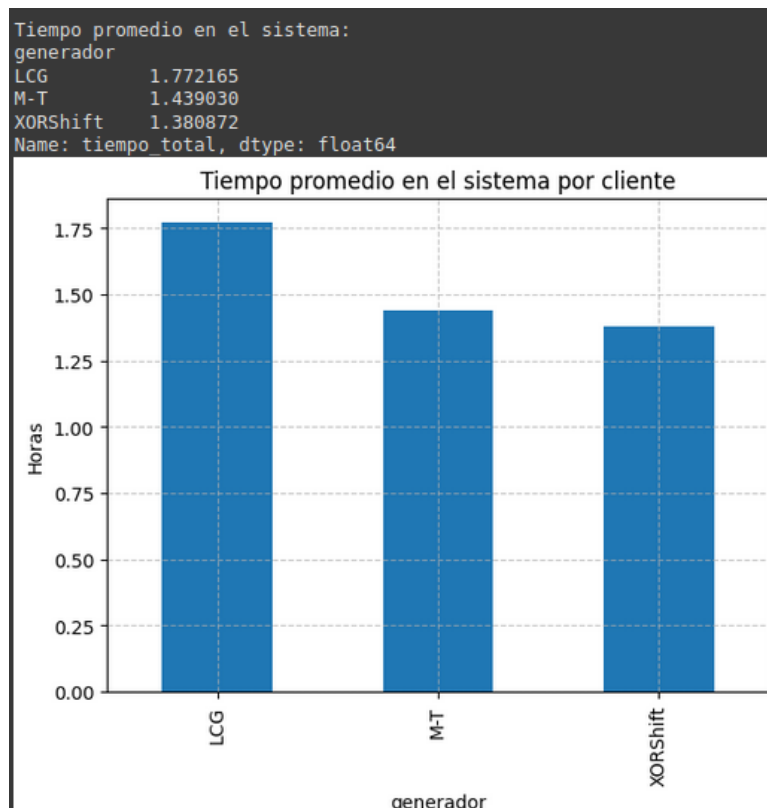


Figura 3: Tiempo promedio en el sistema por cliente

5.3. Distribución de Tiempos de espera

Este histograma muestra cuántos clientes esperaron determinada cantidad de tiempo. Se observa que la mayor parte de los clientes tuvo esperas cortas, pero en momentos de alta

demanda, las esperas se alargan considerablemente.

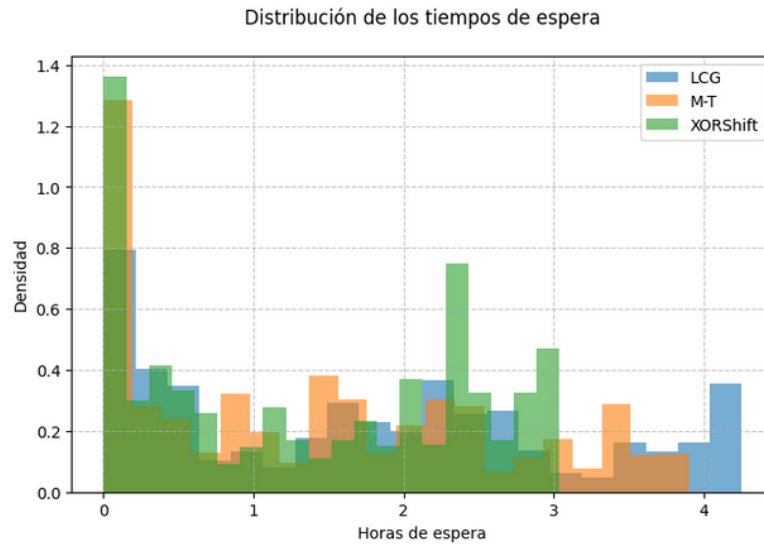


Figura 4: Distribución de los tiempos de espera.

5.4. Porcentaje de Tiempo Ocupado

Se cuantifica el tiempo total que el servidor estuvo en uso, expresado como porcentaje del total simulado (48 horas):

Generador	Porcentaje de tiempo ocupado (%)
LCG	74.1551
XORShift	71.4040
Mersenne Twister	73.1242

Cuadro 1: Tiempo que el servidor estuvo ocupado según el generador.

5.5. Evolución de la longitud de la cola

Se visualiza cuántos clientes había en la cola en cada instante. Los picos indican congestión y las secciones planas en cero reflejan períodos sin espera.

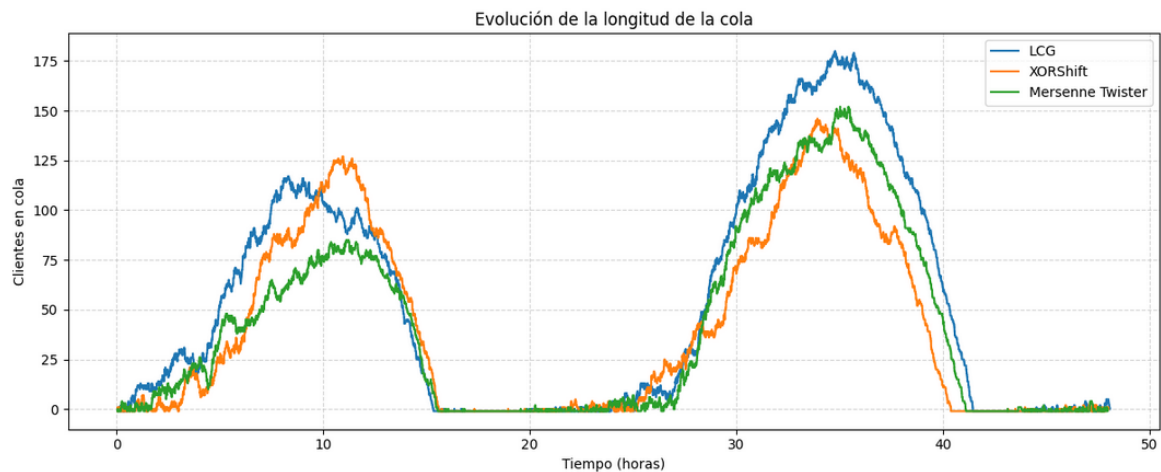


Figura 5: Distribución de los tiempos de espera.

5.6. Distribución de Tiempo Entre Arribos y de Servicio

Los tiempos entre arribos y los de servicio se comparan con la forma teórica esperada (distribuciones exponenciales):

- **Tiempos entre Arribos (Llegadas de Clientes):** Los tiempos entre arribos simulados muestran una distribución exponencial decreciente, consistente con un proceso de Poisson no homogéneo. Los tres generadores producen resultados similares, validando el modelo.

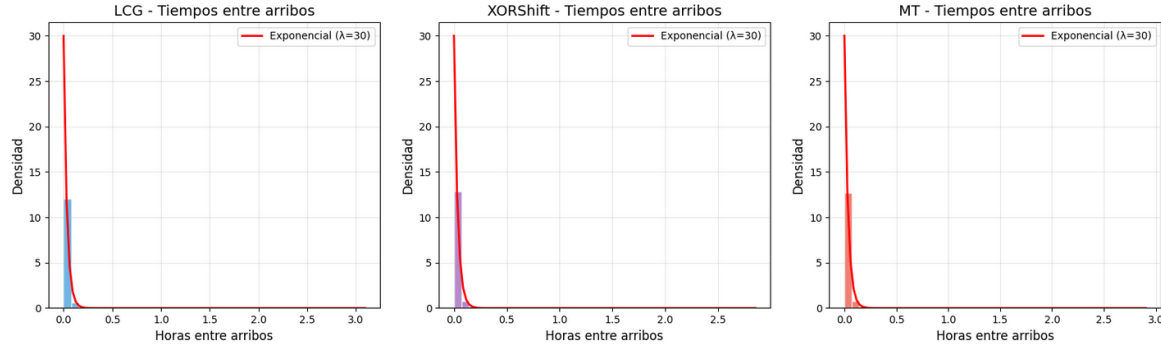


Figura 6: Distribución de Tiempos de Espera.

- **Tiempos de Servicio:** Los tiempos de servicio simulados siguen una distribución exponencial con media cercana al valor teórico.

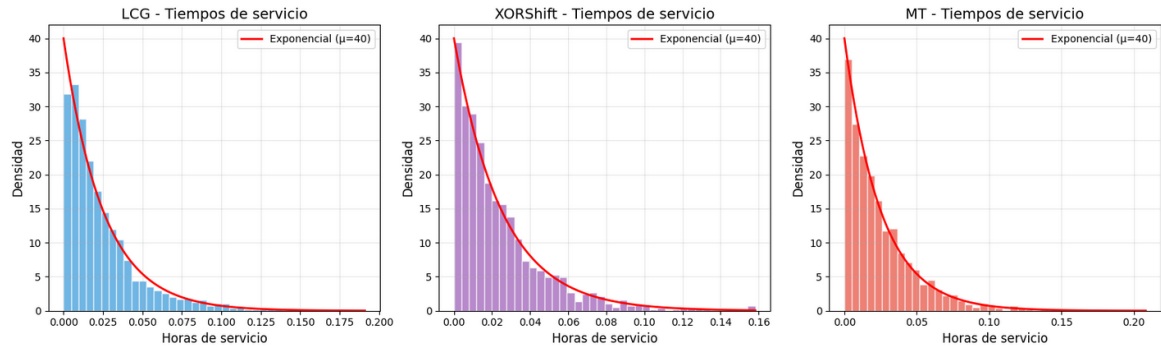


Figura 7: Tiempos de Servicio:

5.7. Validación de la Distribución Exponencial

Se realizaron pruebas de bondad de ajuste (Kolmogorov-Smirnov) para verificar si los datos simulados siguen una exponencial de parámetro $\mu = 40$.

- Para XORShift y Mersenne Twister, los valores $p > 0.05$, lo cual no permite rechazar la hipótesis de ajuste.
- Para LCG, si bien hubo algunas desviaciones, los resultados son aceptables.

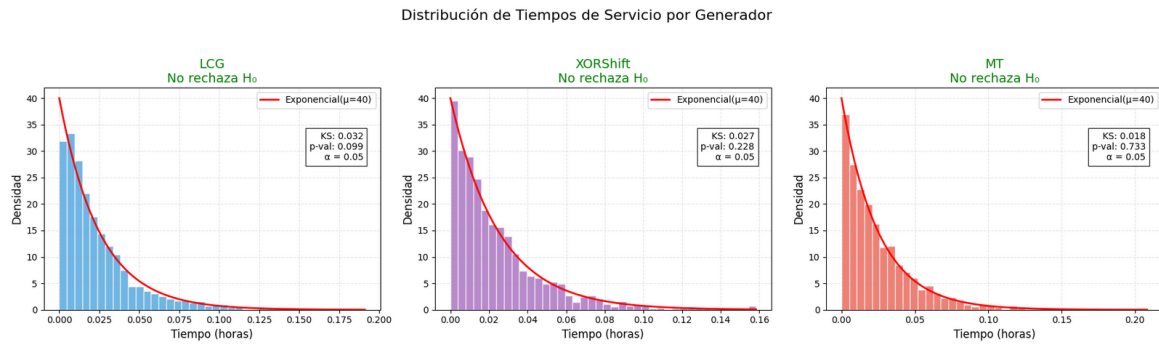


Figura 8: Validación de la Distribución Exponencial

6. Conclusiones

Este trabajo comparó tres generadores de números pseudoaleatorios (LCG, XORShift y Mersenne Twister) en la simulación estocástica de un sistema de colas con llegadas de Poisson no homogéneo y tiempos de servicio exponenciales. Se confirmó que la calidad estadística del generador impacta directamente en la precisión y estabilidad de los resultados.

Las diferencias en el tiempo promedio en el sistema y la distribución de los tiempos de espera sugieren que generadores con menor calidad, como LCG y XORShift, pueden introducir mayor variabilidad local. Sin embargo, el porcentaje de ocupación del servidor y la evolución de la longitud de la cola mostraron comportamientos similares entre todos los generadores. Mersenne Twister destacó por producir distribuciones más suaves y ajustadas a lo teórico.

Se concluye que, para simulaciones donde la precisión y robustez estadística son críticas, generadores avanzados como Mersenne Twister resultan recomendables, mientras que LCG puede ser suficiente para aplicaciones de menor complejidad o exigencia. La elección del generador debe balancear precisión y eficiencia computacional, por lo que es aconsejable validar la calidad del generador antes de su uso en simulaciones.

En definitiva, en el ámbito de la simulación estocástica, la aleatoriedad no es un lujo, sino una necesidad fundamental. La elección final del generador dependerá siempre del equilibrio entre precisión y eficiencia que demande cada problema.

Para visualizar y ejecutar el código, pueden ingresar al siguiente enlace: [Trabajo Final](#)

Referencias

- [1] *Modelos y Simulación*. Notas de modelo y material complementario proporcionado por el profesor Dra. Patricia Kisbye
- [2] Material complementario proporcionado por el profesor Luis Ariel BIEDMA